

# Actividad 5 — El Mono: juego de plataformas

**Duración:** 50 min · **Modalidad:** individual · **Entrega:** repo GitHub público

## 1. Objetivos

- Implementar **gravedad y salto** para un personaje.
- Mover el personaje sobre **plataformas** con colisiones.
- Implementar **objetivos coleccionables** (bananas).
- Comprender la base de los juegos de plataformas (Mario, Sonic, etc.).

## 2. Marco teórico

En un juego de plataformas el personaje vive en un mundo donde **la gravedad siempre lo empuja hacia abajo**. En cada cuadro:

1. La velocidad vertical suma la gravedad: `vel_y += GRAVEDAD`.
2. La posición se actualiza: `y += vel_y`.
3. Si el personaje **toca una plataforma por arriba**, se detiene (`vel_y = 0`) y puede volver a saltar.

El salto es simplemente darle a `vel_y` un valor **negativo grande** (arriba es “menos y”): `vel_y = -13`. Después la gravedad lo frena y lo baja de nuevo: así se forma el arco del salto.

Un detalle importante: la detección de suelo se hace con `collidirect`, y para evitar que el personaje “atraviese” el piso, comparamos el borde inferior del personaje con el borde superior de la plataforma.

## 3. Paso a paso

```
cd mis-videojuegos
mkdir clase_05_el_mono
```

Escribí `clase_05_el_mono/mono.py`:

```
import sys
import pygame
```

```
pygame.init()
```

```
ANCHO, ALTO = 800, 600
pantalla = pygame.display.set_mode((ANCHO, ALTO))
reloj = pygame.time.Clock()
```

```
GRAVEDAD = 0.5
VEL_MOV = 6
```

```

FUERZA_SALTO = -13

mono = pygame.Rect(100, 300, 40, 40)
vel_x, vel_y = 0, 0
en_piso = False

plataformas = [
    pygame.Rect(0, ALTO - 40, ANCHO, 40),      # piso
    pygame.Rect(200, 450, 180, 25),
    pygame.Rect(450, 360, 180, 25),
    pygame.Rect(600, 250, 180, 25),
]

bananas = [
    pygame.Rect(250, 420, 20, 20),
    pygame.Rect(500, 330, 20, 20),
    pygame.Rect(650, 220, 20, 20),
]

juntas = 0

ejecutando = True
while ejecutando:
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            ejecutando = False
        elif evento.type == pygame.KEYDOWN:
            if evento.key == pygame.K_SPACE and en_piso:
                vel_y = FUERZA_SALTO

    # Movimiento horizontal con teclado
    teclas = pygame.key.get_pressed()
    vel_x = (teclas[pygame.K_RIGHT] - teclas[pygame.K_LEFT]) * VEL_MOV

    # Gravedad y movimiento
    vel_y += GRAVEDAD
    mono.x += vel_x
    mono.y += vel_y

    # Colisión con plataformas
    en_piso = False
    for p in plataformas:
        if mono.colliderect(p) and vel_y > 0 and mono.bottom <= p.top + 1
5:
        mono.bottom = p.top
        vel_y = 0
        en_piso = True

    # Juntar bananas
    for b in bananas[:]:

```

```

        if mono.colliderect(b):
            bananas.remove(b)
            juntas += 1

# COMPLETÁ: si el mono se cae de la pantalla (mono.top > ALTO),
# reiniciá el juego (posición inicial, vel_y = 0, bananas de nuevo).

# Dibujar
pantalla.fill((150, 210, 255))
for p in plataformas:
    pygame.draw.rect(pantalla, (90, 60, 30), p)
pygame.draw.rect(pantalla, (160, 110, 50), mono)
for b in bananas:
    pygame.draw.circle(pantalla, (255, 220, 60), b.center, 10)

pygame.display.set_caption(f"El Mono - Bananas: {juntas}")
pygame.display.flip()
reloj.tick(60)

if len(bananas) == 0:
    ejecutando = False

pygame.quit()
sys.exit()

```

#### 4. Actividad de cierre (para el commit)

1. Completá el bloque COMPLETÁ: si el mono se cae al vacío, reiniciá la posición en (100, 300) y la velocidad en 0.
2. Agregá un **reloj**: mostrará cuántos segundos tardaste en juntar todas las bananas. Pista: segundos = pygame.time.get\_ticks() // 1000.
3. Agregá **enemigos simples** (cajas que se mueven de un lado a otro) que te quiten una vida al tocarte.
4. Commit y push:

```

git add .
git commit -m "Clase 05: el mono con plataformas, bananas y caídas"
git push

```

#### 5. Usando IA en esta clase (prompts sugeridos)

**Prompt 1:** “Explicame por qué el salto se hace asignando un valor negativo a vel\_y y no moviendo directamente al personaje.”

**Prompt 2:** “Mi mono cae a través del piso. Mi código: [pegá el código]. ¿Qué condición de colisión me falta?”

**Prompt 3:** “Agregá un enemigo que patrulle una plataforma de izquierda a derecha y reinicie al jugador si lo toca.”

**Prompt 4:** “Dame ideas de 3 niveles distintos para este juego de plataformas, con posiciones de plataformas y bananas.”

## 6. Desafíos para seguir practicando

- Cambiar el mono por una **imagen/sprite** (`pygame.image.load`).
- Que las plataformas **desaparezcan** al pisarlas una vez.
- Agregar una meta (una bandera) que gana el nivel.
- Sonido al juntar bananas (con `pygame.mixer`).

## 7. Rúbrica de evaluación

| Criterio           | Logrado             | En proceso           | No logrado |
|--------------------|---------------------|----------------------|------------|
| Salto y gravedad   | Realistas           | Rígidos              | No salta   |
| Plataformas        | Se para sobre ellas | Atraviesa            | No hay     |
| Bananas            | Las junta y cuenta  | Junta pero no cuenta | No         |
| Reinicio por caída | Funciona            | Fallas               | Falta      |